

Ring-LWE-Based Encrypted Controller With Unlimited Number of Recursive Multiplications and Effect of Error Growth

Yeongjun Jang , Joowon Lee , Seonhong Min , Hyesun Kwak , Junsoo Kim , and Yongsoo Song 

Abstract—In this article, we propose an encrypted dynamic controller that executes an unlimited number of recursive homomorphic multiplications on a Ring Learning With Errors (Ring-LWE)-based cryptosystem without bootstrapping. The proposed controller exhibits lower computational complexity compared to existing encrypted controllers implemented on LWE-based schemes due to the polynomial structure of Ring-LWE. However, the structural difference introduces additional difficulties in analyzing the effect of error growth; Ring-LWE-based schemes inject multiple error coefficients when encrypting a single message, which accumulate under recursive homomorphic multiplications. We show that their effect on control performance can be arbitrarily bounded by the closed-loop stability, thus recovering the performance of the unencrypted controller. Furthermore, a novel method to “pack” a vector into a polynomial is presented, which enhances computational and memory efficiency when applied to the proposed encrypted controller. The effectiveness of the proposed design is demonstrated through numerical simulations.

Index Terms—Encrypted control, homomorphic encryption (HE), networked control systems, privacy, security.

I. INTRODUCTION

THERE is a growing emphasis on securing networked control systems from cyberattacks that could cause significant physical damage [1]. The urgency of this matter has been amplified by the recent development of attack methodologies that exploit disclosed control signals and/or model knowledge

to avoid detection [2]. One of the initial countermeasures was to encrypt the control signals before transmission, which are then decrypted at the controller for computation. However, since the controller resides at the cyber layer in networked control systems, the decrypted signals and control parameters were still exposed to adversaries. In this context, the notion of encrypted control [3], [4], [5] has been introduced. Encrypted controllers utilize homomorphic encryption (HE) to operate directly over encrypted data without decryption, thereby enabling all control signals and parameters to remain encrypted throughout the cyber layer.

When running dynamic controllers over encrypted data, it is essential to account for the effect of error growth caused by the recursive update of the (encrypted) state. This is because most HE schemes that support both addition and multiplication over encrypted data are based on the Learning With Errors (LWE) problem [6]. LWE-based schemes inject errors during encryption to ensure security, which may accumulate and contaminate the message part under recursive homomorphic operations. The bootstrapping technique [7] can be used to refresh these errors, but it is not yet considered practical for real-time systems.

In [8], the effect of error growth has been addressed from a system-theoretic perspective. It has been shown that the closed-loop stability of the plant and the predesigned (unencrypted) controller can suppress the effect of error growth on control performance, even when the encrypted controller performs recursive multiplications on an LWE-based scheme. However, the multiplication technique [9] used in [8] demands high computational resources and substantial memory usage.

In fact, there is an algebraic variant of LWE, called Ring-LWE [10], that is more efficient in terms of computation and memory, while offering the same level of security. In this regard, widely used HE libraries, such as Microsoft SEAL [11], HElib [12], and Lattigo [13], are built upon Ring-LWE-based cryptosystems. Such efficiency comes from their structure based on polynomial rings; however, this also introduces further challenges regarding the effect of error growth. Whereas LWE-based schemes inject errors as scalars, Ring-LWE-based schemes inject errors as polynomials, where each coefficient is a scalar error, and these error coefficients accumulate and affect each other under recursive multiplications (see [12]).

Received 24 April 2025; accepted 12 June 2025. Date of publication 26 June 2025; date of current version 29 December 2025. This work was supported by the National Research Foundation of Korea under Grant RS-2022-00165417, Grant RS-2023-00211649, and Grant RS-2024-00353032 funded by the Korea Government (Ministry of Science and ICT). Recommended by Associate Editor Y. Wang. (Corresponding author: Junsoo Kim.)

Yeongjun Jang and Joowon Lee are with the ASRI, Department of Electrical and Computer Engineering, Seoul National University, Seoul 08826, South Korea (e-mail: jangyj@cde.snu.ac.kr; jwlee@cde.snu.ac.kr).

Seonhong Min, Hyesun Kwak, and Yongsoo Song are with the Department of Computer Science and Engineering, Seoul National University, Seoul 08826, South Korea (e-mail: minsh@snu.ac.kr; hskwak@snu.ac.kr; y.song@snu.ac.kr).

Junsoo Kim is with the Department of Electrical and Information Engineering, Seoul National University of Science and Technology, Seoul 01811, South Korea (e-mail: junsookim@seoultech.ac.kr).

Digital Object Identifier 10.1109/TCNS.2025.3583610

Recent studies [14], [15] on utilizing Ring-LWE-based cryptosystems to encrypt dynamic controllers have avoided recursive homomorphic multiplications with the help of re-encryption, which requires an additional communication line between the plant and the controller. To the best of our knowledge, *a method to perform an unlimited number of recursive homomorphic multiplications on a Ring-LWE-based cryptosystem without bootstrapping has not yet been reported.*

In this article, we propose an encrypted dynamic controller that executes an unlimited number of recursive multiplications on a Ring-LWE-based cryptosystem without bootstrapping. In the proposed design, only the constant terms of the error polynomials affect control performance. It is shown that their effects can be arbitrarily bounded by means of closed-loop stability, even when the *noneffective* higher-order terms of the error polynomials overflow the bounded message space of the Ring-LWE-based scheme. For multiplications, we employ the external product of [16], called the “Ring Gentry–Sahai–Waters (Ring-GSW) scheme,” which offers lower computational complexity than the multiplication technique on LWE-based schemes used in [8]. The proposed encrypted controller’s ability to perform recursive multiplications obviates the need for re-encryption, unlike the previous results [14] and [15].

Furthermore, we propose and apply a novel method to encode a vector into a polynomial—such methods are called “packing” in cryptography—that accelerates computation and improves memory efficiency by replacing multiple scalar operations with a single polynomial operation. We present an encrypted controller equipped with the packing method that preserves the ability to perform an unlimited number of recursive homomorphic multiplications without bootstrapping. The efficiency of utilizing the packing algorithm is analyzed and demonstrated through numerical simulations.

The rest of this article is organized as follows. Section II introduces a Ring-LWE-based scheme and formulates the problem. Section III presents the proposed encrypted dynamic controller, and Section IV describes and applies the proposed packing algorithm. Section V provides simulation results. Finally, Section VI concludes this article.

Notations: The set of integers, nonnegative integers, rational numbers, and real numbers are denoted by $\mathbb{Z}$, $\mathbb{N}$, $\mathbb{Q}$, and $\mathbb{R}$, respectively. The floor, ceiling, and rounding operations are denoted by $\lfloor \cdot \rfloor$, $\lceil \cdot \rceil$, and $\lceil \cdot \rceil$, respectively. For $q \in \mathbb{N}$, we define $\mathbb{Z}_q := \mathbb{Z} \cap [-q/2, q/2)$. For $a \in \mathbb{Z}$ and $q \in \mathbb{N}$, we define the modulo operation by $a \bmod q := a - \lfloor (a + q/2)/q \rfloor q \in \mathbb{Z}_q$. The floor, ceiling, rounding, and modulo operations are defined componentwisely for vectors and matrices, and coefficientwisely for polynomials. For a (matrix) vector of scalars, $\|\cdot\|$ denotes the (induced) infinity norm. For a polynomial $a = \sum_{i=0}^{N-1} a_i X^i$, let $\|a\| := \max_{0 \leq i < N} \{|a_i|\}$. For a matrix of polynomials with m -rows and n -columns denoted by $A = \{A_{i,j}(X)\}$, we define $\|A\| := \max_{1 \leq i \leq m} \sum_{j=1}^n \|A_{i,j}(X)\|$ and $A|_{X=0} := \{A_{i,j}(0)\}$. Let $0_{m \times n} \in \mathbb{Z}^{m \times n}$ and $I_n \in \mathbb{Z}^{n \times n}$ denote the zero matrix and the identity matrix, respectively. The Kronecker product is denoted by $\otimes$. For a sequence $v_1, \dots, v_n$

of scalars or column vectors, we define $[v_1; \dots; v_n] := [v_1^\top \dots v_n^\top]^\top$.

II. PRELIMINARIES AND PROBLEM FORMULATION

A. Ring-LWE-Based Cryptosystem

This subsection explains the Ring-LWE-based cryptosystem [10], [16] used in this article. The cryptosystem involves two encryption algorithms, namely, Ring-LWE encryption and Ring-GSW encryption. We will describe each encryption algorithm and illustrate how to perform additions and multiplications over encrypted data.

To proceed, some definitions are introduced. For $N \in \mathbb{N}$, let $\mathcal{R} := \mathbb{Z}[X]/\langle X^N + 1 \rangle$ be the ring of integer polynomials modulo $X^N + 1$, whose elements are represented by integer polynomials of degree less than N . The residue ring of $\mathcal{R}$ modulo $q \in \mathbb{N}$ is denoted by $\mathcal{R}_q$. The elements of $\mathcal{R}_q$ are polynomials in $\mathcal{R}$ whose coefficients belong to the set $\mathbb{Z}_q$. The addition and multiplication of $a = \sum_{i=0}^{N-1} a_i X^i \in \mathcal{R}_q$ and $b = \sum_{i=0}^{N-1} b_i X^i \in \mathcal{R}_q$ are defined by

$$\begin{aligned} a + b &:= \sum_{i=0}^{N-1} (a_i + b_i) X^i \bmod q \\ a \cdot b &:= \sum_{i=0}^{N-1} c_i X^i \bmod q \end{aligned} \quad (1)$$

where $c_i := \sum_{j=0}^i a_j b_{i-j} - \sum_{j=i+1}^{N-1} a_j b_{N+i-j}$. These operations can be interpreted as general addition and multiplication of real polynomials, where X^N is regarded as -1 and the coefficients are mapped to the set $\mathbb{Z}_q$ via the modulo operation. Although the modulo operation is embedded in the definitions of (1), we often deliberately indicate it for readers who are not familiar with operations on polynomial rings.

Now, we describe the Ring-LWE-based scheme of [10]. Ciphertexts (encrypted data) are written in bold type to distinguish them from plaintexts (messages to be encrypted).

- 1) (Setup) Choose a power of two $N \in \mathbb{N}$ and a prime $q \in \mathbb{N}$ such that $1 = q \bmod 2N$. Fix an error distribution ψ over $\mathcal{R}$ bounded by $\sigma > 0$, i.e., any $e \in \mathcal{R}$ sampled according to ψ is bounded as $\|e\| \leq \sigma$. Generate the secret key $\mathbf{sk} \in \mathcal{R}_q$ by sampling each of its coefficient uniformly at random from $\{-1, 0, 1\}$.
- 2) (Ring-LWE encryption) $\text{Enc} : \mathcal{R}_q \rightarrow \mathcal{R}_q^2$. Sample a polynomial $a \in \mathcal{R}_q$ uniformly at random from $\mathcal{R}_q$ and an error polynomial e according to ψ . Then, the Ring-LWE encryption of a plaintext $m \in \mathcal{R}_q$ is defined by

$$\text{Enc}(m) := \begin{bmatrix} \mathbf{sk} \cdot a + m + e \\ a \end{bmatrix} \bmod q \in \mathcal{R}_q^2.$$

- 3) (Decryption) $\text{Dec} : \mathcal{R}_q^2 \rightarrow \mathcal{R}_q$. The decryption of a ciphertext $\mathbf{c} \in \mathcal{R}_q^2$ is defined by

$$\text{Dec}(\mathbf{c}) := [1 \quad -\mathbf{sk}] \mathbf{c} \bmod q \in \mathcal{R}_q.$$

- 4) (Homomorphic addition) $\boxplus : \mathcal{R}_q^2 \times \mathcal{R}_q^2 \rightarrow \mathcal{R}_q^2$. The addition of $\mathbf{c}_1 \in \mathcal{R}_q^2$ and $\mathbf{c}_2 \in \mathcal{R}_q^2$ is defined by

$$\mathbf{c}_1 \boxplus \mathbf{c}_2 := \mathbf{c}_1 + \mathbf{c}_2 \bmod q \in \mathcal{R}_q^2.$$

The security of the scheme is ensured by the hardness of the Ring-LWE problem [10]. Roughly, distinguishing $\text{Enc}(m)$ from a randomly generated vector in $\mathcal{R}_q^2$ is “difficult” without the secret key, provided that N is sufficiently large. In fact, there are no better attacks available against Ring-LWE-based schemes than against LWE-based schemes [17].

The following proposition is immediate from the definitions.

Proposition 1: The following properties hold for any $m \in \mathcal{R}_q$, $c \in \mathcal{R}_q^2$, and $c' \in \mathcal{R}_q^2$.

- 1) $\text{Dec}(\text{Enc}(m)) = m + e \bmod q$ for some $e \in \mathcal{R}$ such that $\|e\| \leq \sigma$.
- 2) $\text{Dec}(c \boxplus c') = \text{Dec}(c) + \text{Dec}(c') \bmod q$.
- 3) $\text{Dec}(m \cdot c) = m \cdot \text{Dec}(c) \bmod q$. $\square$

Property 2) of Proposition 1 is known as the additively homomorphic property. Along with property 1), it signifies how the injected errors accumulate under homomorphic additions; suppose that $e_1 \in \mathcal{R}$ and $e_2 \in \mathcal{R}$ are the errors injected during the encryption of $m_1 \in \mathcal{R}_q$ and $m_2 \in \mathcal{R}_q$, respectively. Then, it follows that:

$$\text{Dec}(\text{Enc}(m_1) \boxplus \text{Enc}(m_2)) = (m_1 + m_2) + (e_1 + e_2) \bmod q$$

where the errors are added together.

Remark 1: In practice, an encrypted message can be recovered without error as follows: For $m \in \mathcal{R}_q$ and $c \in \mathcal{R}_q^2$, let Enc and Dec be modified as $\text{Enc}_L(m) := \text{Enc}(m/L \bmod q)$ and $\text{Dec}_L(c) := \lceil L \cdot \text{Dec}(c) \rceil \bmod q$, respectively, where $1/L \in \mathbb{N}$ is some scale factor. If L is chosen sufficiently small such that $1/L > 2\sigma$, then

$$\text{Dec}_L(\text{Enc}_L(m)) = \lceil L \cdot (m/L + e \bmod q) \rceil = m$$

as long as $\|m\| < Lq/2 - 1/2$. $\square$

Next, we introduce the Ring-GSW encryption [16] and describe a method to perform multiplications over encrypted data, referred to as the “external product.” Here, we apply the “special modulus” technique [18] to reduce the growth of errors under the external product.

- 1) (Setup) Choose a power of two $\nu \in \mathbb{N}$ and let $d := \lceil \log_\nu q \rceil$. Choose a prime $P \in \mathbb{N}$ as the “special modulus.”
- 2) (Ring-GSW encryption) $\text{Enc}' : \mathcal{R}_q \rightarrow \mathcal{R}_{qP}^{2 \times 2d}$. The Ring-GSW encryption of a plaintext $M \in \mathcal{R}_q$ is defined by

$$\text{Enc}'(M) := PM \cdot \mathcal{G} + \mathcal{L} \bmod qP \in \mathcal{R}_{qP}^{2 \times 2d}$$

where $\mathcal{G} := \begin{bmatrix} 1 & \nu & \dots & \nu^{d-1} \end{bmatrix} \otimes I_2 \in \mathbb{Z}^{2 \times 2d}$ and $\mathcal{L} := [\text{Enc}(0) \dots \text{Enc}(0)] \in \mathcal{R}_{qP}^{2 \times 2d}$.

The external product multiplies a Ring-LWE ciphertext (which belongs to $\mathcal{R}_q^2$) with a Ring-GSW ciphertext (which belongs to $\mathcal{R}_{qP}^{2 \times 2d}$) and returns a Ring-LWE ciphertext. For the external product, the Ring-LWE ciphertext is first decomposed into a vector in $\mathcal{R}_q^{2d}$ having “small” elements.

- 1) (Ring-LWE ciphertext decomposition) $D : \mathcal{R}_q^2 \rightarrow \mathcal{R}_q^{2d}$. For $c \in \mathcal{R}_q^2$, compute $\bar{c}_i \in \mathcal{R}_q^2$ for $i = 0, 1, \dots, d-1$ such that $c = \sum_{i=0}^{d-1} \nu^i \cdot \bar{c}_i$ and $\|\bar{c}_i\| \leq \nu/2$. The decomposition of $c \in \mathcal{R}_q^2$ is defined by

$$D(c) := [\bar{c}_0; \dots; \bar{c}_{d-1}] \in \mathcal{R}_q^{2d}.$$

- 2) (External product) $\boxtimes : \mathcal{R}_{qP}^{2 \times 2d} \times \mathcal{R}_q^2 \rightarrow \mathcal{R}_q^2$. The external product between $C \in \mathcal{R}_{qP}^{2 \times 2d}$ and $c \in \mathcal{R}_q^2$ is defined by

$$C \boxtimes c := \lceil P^{-1} \cdot (CD(c) \bmod qP) \rceil \bmod q \in \mathcal{R}_q^2.$$

The following proposition states the multiplicatively homomorphic property of the external product.

Proposition 2: For any $M \in \mathcal{R}_q$ and $c \in \mathcal{R}_q^2$, we have

$$\text{Dec}(\text{Enc}'(M) \boxtimes c) = M \cdot \text{Dec}(c) + \Delta \bmod q \quad (2)$$

for some $\Delta \in \mathcal{R}_q$ such that

$$\|\Delta\| \leq P^{-1}dN\sigma\nu + \frac{1}{2}(N+1) =: \sigma_{\text{Mult}}.$$

$\square$

Proof: By the definitions of Enc' and $\boxtimes$, we have

$$\text{Enc}'(M) \boxtimes c = \lceil P^{-1} \cdot ((PM \cdot \mathcal{G} + \mathcal{L})D(c) \bmod qP) \rceil \bmod q.$$

Since $(PM \cdot \mathcal{G} + \mathcal{L})D(c) \bmod qP = (PM \cdot \mathcal{G} + \mathcal{L})D(c) + qP \cdot k$ for some $k \in \mathcal{R}^2$ and $\mathcal{G}D(c) = c$, it follows that:

$$\begin{aligned} \text{Dec}(\text{Enc}'(M) \boxtimes c) &= \lceil 1 \quad -sk \rceil [M \cdot c + P^{-1} \cdot \mathcal{L}D(c) + q \cdot k] \bmod q \\ &= \lceil 1 \quad -sk \rceil (M \cdot c + q \cdot k + \lceil P^{-1} \cdot \mathcal{L}D(c) \rceil) \bmod q \\ &= M \cdot \text{Dec}(c) + P^{-1} \cdot \lceil 1 \quad -sk \rceil \mathcal{L}D(c) \\ &\quad + \lceil 1 \quad -sk \rceil (\lceil P^{-1} \cdot \mathcal{L}D(c) \rceil - P^{-1} \cdot \mathcal{L}D(c)) \bmod q. \end{aligned}$$

Note that $\lceil 1 \quad -sk \rceil \mathcal{L}$ is a $2d$ -dimensional row vector of polynomials bounded as $\|\lceil 1 \quad -sk \rceil \mathcal{L}\| \leq 2d\sigma$. Since $D(c)$ is also bounded as $\|D(c)\| \leq \nu/2$, applying (1) yields $\|\lceil 1 \quad -sk \rceil \mathcal{L}D(c)\| \leq P^{-1}dN\sigma\nu$. As $\|sk\| \leq 1$, it again follows from (1) that:

$$\|\lceil 1 \quad -sk \rceil (\lceil P^{-1} \cdot \mathcal{L}D(c) \rceil - P^{-1} \cdot \mathcal{L}D(c))\| \leq \frac{1}{2}(N+1)$$

and this concludes the proof. $\blacksquare$

In (2), the ciphertext c is not necessarily a newly generated output from Enc . Instead, it could be a ciphertext that has already gone through multiple homomorphic operations, thus carrying some accumulated error. Nevertheless, Proposition 2 guarantees that the newborn error under the external product, which is Δ in (2), is bounded by a constant regardless of the error accumulated in c .

For simplicity, we abuse notation and define Enc and Enc' componentwisely with respect to vectors or matrices of plaintexts. The operations $\boxplus$ and Dec are also applied componentwisely to vectors or matrices of ciphertexts. For $K = \{K_{i,j}\} \in \mathcal{R}_q^{h \times l}$ and $c = [c_1; \dots; c_l] \in \mathcal{R}_q^{2l}$, the external product between $\text{Enc}'(K) \in \mathcal{R}_{qP}^{2h \times 2ld}$ and c is defined by

$$\begin{aligned} \text{Enc}'(K) \boxtimes c &= \sum_{j=1}^l [\text{Enc}'(K_{1,j}) \boxtimes c_j; \dots; \text{Enc}'(K_{h,j}) \boxtimes c_j] \bmod q \in \mathcal{R}_q^{2h} \end{aligned}$$

where the summation is taken with respect to the operation $\boxplus$. Then, it follows from Proposition 2 that:

$$\begin{aligned}
& \text{Dec}(\text{Enc}'(K) \boxtimes c) \\
&= \sum_{j=1}^l [K_{1,j} \cdot \text{Dec}(c_j); \dots; K_{h,j} \cdot \text{Dec}(c_j)] + \Delta \bmod q \\
&= K \cdot \text{Dec}(c) + \Delta \bmod q \in \mathcal{R}_q^h
\end{aligned} \tag{3}$$

for some $\Delta \in \mathcal{R}_q^h$ such that $\|\Delta\| \leq l \cdot \sigma_{\text{Mult}}$.

B. Problem Formulation

Consider a discrete-time plant written by

$$\begin{aligned}
x_p(t+1) &= Ax_p(t) + Bu(t), \quad x_p(0) = x_p^{\text{ini}} \\
y(t) &= Cx_p(t)
\end{aligned} \tag{4}$$

where $x_p(t) \in \mathbb{R}^{n_p}$ is the state with the initial value $x_p^{\text{ini}} \in \mathbb{R}^{n_p}$, $u(t) \in \mathbb{R}^m$ is the input, and $y(t) \in \mathbb{R}^p$ is the output. To stabilize (4), assume that a controller has been designed as

$$\begin{aligned}
x(t+1) &= Fx(t) + Gy(t), \quad x(0) = x^{\text{ini}} \\
u(t) &= Hx(t)
\end{aligned} \tag{5}$$

where $x(t) \in \mathbb{R}^n$ is the state with the initial value $x^{\text{ini}} \in \mathbb{Q}^n$, and the control parameters are given as

$$F \in \mathbb{Z}^{n \times n}, \quad G \in \mathbb{Q}^{n \times p}, \quad \text{and} \quad H \in \mathbb{Q}^{m \times n}.$$

It has been studied in [8] that the state matrix, such as F of (5), needs to be an integer matrix, in order to implement a dynamic system over encrypted data. Subsequently, there have been studies on designing stabilizing controllers with integer state matrices [21] or on converting a given controller to have an integer state matrix, while preserving its control performance [19], [20].¹ Throughout this article, we assume that such conversion has been conducted a priori. Moreover, the elements of $\{x^{\text{ini}}, G, H\}$ being rational numbers are fairly reasonable because real numbers can be approximated to rational numbers with arbitrary precision, as $\mathbb{Q}$ is dense in $\mathbb{R}$.

The objective of this article is to implement (5) as an encrypted controller, which satisfies the following.

- 1) The encrypted controller performs an unlimited number of recursive homomorphic multiplications, on the Ring-LWE-based cryptosystem described in Section II-A.
- 2) It operates for an infinite-time horizon, without bootstrapping or any other methods to refresh the (encrypted) state.
- 3) The performance of the proposed controller is equivalent to that of (5) in the following sense: Let $u(t)$ be the control input decrypted from the encrypted controller and $u^{\text{nom}}(t)$ be the input from the predesigned controller (5). Given $\epsilon > 0$

$$\|u(t) - u^{\text{nom}}(t)\| \leq \epsilon \tag{6}$$

holds for all $t \geq 0$.

III. ENCRYPTED CONTROLLER

A method to operate the controller (5) using the Ring-LWE-based scheme is presented. The effect of error growth is suppressed by the stability of the closed-loop system (4) with (5), enabling the proposed encrypted controller to execute an unlimited number of recursive homomorphic multiplications without the use of bootstrapping. In addition, we provide an explicit guideline for choosing the encryption parameters to achieve (6).

A. Encrypted Controller Design

We implement the operations of (5) over encrypted data utilizing the homomorphic properties stated in Propositions 1 and 2. To this end, each component of the matrices and vectors in (5) is converted into a constant polynomial in $\mathcal{R}_q$, which is the plaintext space of the Ring-LWE-based cryptosystem.

During the offline procedure, the control parameters $\{F, G, H\}$ and the initial value x^{ini} of (5) are quantized and encrypted as follows; as the elements of G and H belong to $\mathbb{Q}$, a scale factor $1/s \in \mathbb{N}$ can be chosen:

$$\frac{G}{s} \in \mathbb{Z}^{n \times p}, \quad \frac{H}{s} \in \mathbb{Z}^{m \times n}.$$

We also introduce a scale factor $1/L \in \mathbb{N}$, in order to deal with the effect of errors, as illustrated in Remark 1. Then, the control parameters and the initial value are encrypted as

$$\begin{aligned}
\mathbf{F} &:= \text{Enc}'(F), & \mathbf{G} &:= \text{Enc}'(G/s) \\
\mathbf{H} &:= \text{Enc}'(H/s), & \mathbf{x}^{\text{ini}} &:= \text{Enc}(\lceil x^{\text{ini}}/r \rceil / (sL))
\end{aligned} \tag{7}$$

where $r > 0$ is the quantization step size at the sensor. We encrypt the control parameters as matrices of Ring-GSW ciphertexts and the initial state as a vector of Ring-LWE ciphertexts. This is because the external product between a Ring-GSW ciphertext and a Ring-LWE ciphertext returns a Ring-LWE ciphertext, as the product between a matrix and a vector returns a vector.

At each time step $t \geq 0$ of the online procedure, the sensor quantizes the plant output $y(t)$ as

$$\bar{y}(t) := \left\lfloor \frac{y(t)}{r} \right\rfloor \in \mathbb{Z}^p$$

and then encrypts it as

$$\mathbf{y}(t) := \text{Enc}(\bar{y}(t)/L) \in \mathcal{R}_q^{2p}.$$

Then, $\mathbf{y}(t)$ is sent to the encrypted controller, designed as

$$\begin{aligned}
\mathbf{x}(t+1) &= (\mathbf{F} \boxtimes \mathbf{x}(t)) \boxplus (\mathbf{G} \boxtimes \mathbf{y}(t)), \quad \mathbf{x}(0) = \mathbf{x}^{\text{ini}} \\
\mathbf{u}(t) &= \mathbf{H} \boxtimes \mathbf{x}(t)
\end{aligned} \tag{8a}$$

where $\mathbf{x}(t) \in \mathcal{R}_q^{2n}$ is the state and $\mathbf{u}(t) \in \mathcal{R}_q^{2m}$ is the output. The output $\mathbf{u}(t)$ is transmitted to the actuator, decrypted, and then scaled down to obtain the plant input $u(t)$, as

$$u(t) = rs^2L \cdot \text{Dec}(\mathbf{u}(t))|_{X=0}. \tag{8b}$$

B. Performance Analysis

Here, we analyze the performance of the proposed encrypted controller (8) and show that (6) holds with an appropriate choice

¹The conversion is always possible through approximation [19]. Other results [20], [21] avoid such approximation under some systematic assumptions.

of the parameters $\{q, r, s, L\}$. Specifically, we examine the effect of error growth and show that: 1) the error coefficients in the higher-order terms do not affect control performance and 2) the effect of the error coefficients in the constant terms remain bounded by the stability of the closed-loop system (4) with (5) in the proposed design.

For analysis, we first define

$$\begin{aligned} u(t) &:= rs^2L \cdot \text{Dec}(u(t)) = \sum_{i=0}^{N-1} u_i(t)X^i \\ x(t) &:= rsL \cdot \text{Dec}(x(t)) = \sum_{i=0}^{N-1} x_i(t)X^i \end{aligned} \quad (9)$$

where $u_i(t) \in \mathbb{R}^m$ and $x_i(t) \in \mathbb{R}^n$ are the vectors consisting of the i -th coefficients of $u(t)$ and $x(t)$, respectively, for $i = 0, 1, \dots, N-1$. Note that $u_0(t) \equiv u(t)$ of (8b), and we interchangeably use both notations in this section.

Next, we define some “lumped errors” that are caused by quantization and encryption algorithms, written by

$$\begin{aligned} e^x(t) &:= x(t+1) - (Fx(t) + Gy(t)), \quad e^{\text{ini}} := x(0) - x^{\text{ini}} \\ e^u(t) &:= u(t) - Hx(t). \end{aligned} \quad (10)$$

Likewise, the vectors consisting of the i -th coefficients of $e^x(t)$, $e^u(t)$, and e^{ini} are denoted by $e_i^x(t) \in \mathbb{R}^n$, $e_i^u(t) \in \mathbb{R}^m$, and $e_i^{\text{ini}} \in \mathbb{R}^n$, respectively, for $i = 0, 1, \dots, N-1$.

Then, the dynamics of each coefficient vector $x_i(t)$ can be derived from (10), as

$$\begin{aligned} x_0(t+1) &= Fx_0(t) + Gy(t) + e_0^x(t), \quad x_0(0) = x^{\text{ini}} + e_0^{\text{ini}} \\ u_0(t) &= Hx_0(t) + e_0^u(t) \end{aligned} \quad (11)$$

and for $i = 1, \dots, N-1$,

$$\begin{aligned} x_i(t+1) &= Fx_i(t) + e_i^x(t), \quad x_i(0) = e_i^{\text{ini}} \\ u_i(t) &= Hx_i(t) + e_i^u(t). \end{aligned}$$

The reason why the dynamics of $x_0(t)$ differs from those of other $x_i(t)$ s is that the components of $y(t)$ and x^{ini} are scalars, and thus, only affect $x_0(t)$.

Since the output $u(t) = u_0(t)$ of (8b) is solely determined from (11), the closed-loop system (4) with (8) can be divided into two parts:

$$\begin{aligned} \begin{bmatrix} x_1(t+1) \\ \vdots \\ x_{N-1}(t+1) \end{bmatrix} &= I_{N-1} \otimes F \begin{bmatrix} x_1(t) \\ \vdots \\ x_{N-1}(t) \end{bmatrix} + \begin{bmatrix} e_1^x(t) \\ \vdots \\ e_{N-1}^x(t) \end{bmatrix} \\ x_i(0) &= e_i^{\text{ini}}, \quad i = 1, \dots, N-1 \end{aligned} \quad (12)$$

which consists of coefficients in the higher-order terms, and

$$\begin{aligned} \begin{bmatrix} x_p(t+1) \\ x_0(t+1) \end{bmatrix} &= \underbrace{\begin{bmatrix} A & BH \\ GC & F \end{bmatrix}}_{=: \bar{A}} \begin{bmatrix} x_p(t) \\ x_0(t) \end{bmatrix} + \begin{bmatrix} B & 0 \\ 0 & I_n \end{bmatrix} \begin{bmatrix} e_0^u(t) \\ e_0^x(t) \end{bmatrix} \\ \begin{bmatrix} x_p(0) \\ x_0(0) \end{bmatrix} &= \begin{bmatrix} x_p^{\text{ini}} \\ x^{\text{ini}} + e_0^{\text{ini}} \end{bmatrix} \end{aligned} \quad (13)$$

which is only relevant to the coefficients in the constant terms. This shows that the performance of the encrypted controller (8) is equivalent to that of the controller (11) over $\mathbb{R}$.

The following lemma states that $e_0^x(t)$, $e_0^u(t)$, and e_0^{ini} of (13), which can be thought of as perturbations, remain bounded for all $t \geq 0$ by the closed-loop stability if the modulus q is chosen sufficiently large. To state the lemma, some technical definitions are given. Since the closed-loop state matrix $\bar{A}$ of (13) is Schur stable by the assumption on (5), there exists $M > 0$ and $0 \leq \lambda < 1$ such that $\|\bar{A}^k\| \leq M\lambda^k$ for all $k \geq 0$ [22, Thm. 3.5]. We define the function $\eta : \mathbb{R} \times \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ by

$$\eta(\alpha, \beta, \gamma) := M \left(\left\| \begin{bmatrix} x_p^{\text{ini}} \\ x^{\text{ini}} \end{bmatrix} \right\| + \gamma + \frac{\|B\|\beta + \alpha}{1 - \lambda} \right).$$

Lemma 1: Given the parameters $r > 0$, $1/s \in \mathbb{N}$, and $1/L \in \mathbb{N}$, the perturbations of (13) remain bounded as

$$\begin{aligned} \|e_0^x(t)\| &\leq \alpha(r, s, L) := rsL(n+p)\sigma_{\text{Mult}} + r \frac{\|G\|}{2} + rL\|G\|\sigma \\ \|e_0^u(t)\| &\leq \beta(r, s, L) := rs^2Ln\sigma_{\text{Mult}} \\ \|e_0^{\text{ini}}\| &\leq \gamma(r, s, L) := rsL\sigma + \frac{r}{2} \end{aligned} \quad (14)$$

for all $t \geq 0$ if

$$q > 2 \max \left\{ \frac{\bar{\eta}(r, s, L)}{rsL}, \frac{\|H\|\bar{\eta}(r, s, L) + \beta(r, s, L)}{rs^2L} \right\} \quad (15)$$

where $\bar{\eta}(r, s, L) := \eta(\alpha(r, s, L), \beta(r, s, L), \gamma(r, s, L))$. $\square$

Proof: See Appendix A. $\blacksquare$

Lemma 1 implies that the perturbations $e_0^x(t)$, $e_0^u(t)$, and e_0^{ini} can be arbitrarily bounded by decreasing r , s , and L , because their upper bound functions $\alpha(r, s, L)$, $\beta(r, s, L)$, and $\gamma(r, s, L)$ vanish at the origin. However, it is not guaranteed whether the perturbations $e_i^x(t)$, $e_i^u(t)$, and e_i^{ini} in (12) will remain bounded or grow unboundedly. Interestingly, the following theorem states that the performance error in (6) can be made arbitrarily small with the choice of the parameters $\{q, r, s, L\}$, which implies that we may neglect the perturbations in (12) growing unboundedly.

Theorem 1: For given $\epsilon > 0$, if the parameters $r > 0$, $1/s \in \mathbb{N}$, and $1/L \in \mathbb{N}$ satisfy

$$\beta(r, s, L) \leq \frac{\epsilon}{2} \quad \text{and} \quad \bar{\eta}(r, s, L) \leq M \left\| \begin{bmatrix} x_p^{\text{ini}} \\ x^{\text{ini}} \end{bmatrix} \right\| + \frac{\epsilon}{2\|H\|} \quad (16)$$

and the modulus q satisfies (15), then the encrypted controller (8) guarantees that (6) holds for all $t \geq 0$. $\square$

Proof: See Appendix B. $\blacksquare$

Note that (16) can be satisfied by choosing a sufficiently small r , s , and L . We suggest that r be chosen prior to s and L because decreasing r may require higher sensor resolution in practice, which could be costly. After choosing r , s , and L , the modulus q can be chosen to satisfy (15).

Remark 2: The proposed Ring-LWE-based encrypted controller (8) exhibits lower time complexity compared to the LWE-based encrypted controller presented in [8]. At each time step, both controllers perform $(n^2 + np + nm)$ -external products; however, the time complexities of the external product [16] used in (8) and the external product [9] used in [8] are given by $\mathcal{O}(dN \log N)$ and $\mathcal{O}(dN^2)$, respectively. $\square$

IV. ENCRYPTED CONTROLLER WITH PACKING

In this section, we propose a novel packing algorithm that encodes a vector into a polynomial and present an encrypted controller that integrates the packing algorithm. The encrypted controller equipped with the packing algorithm offers significantly improved computational and memory efficiency compared to the encrypted controller (8), while preserving the ability to perform an unlimited number of recursive homomorphic multiplications without bootstrapping, by means of the closed-loop stability. For notational convenience, we often omit the modulo operation $\text{mod } q$ in this section.

A. Packing and Unpacking Algorithms

First, let us define

$$\tau := 2^{\lceil \log_2(\max\{n, m, p\}) \rceil} \in \mathbb{N}$$

which is the smallest power of two such that $\tau \geq \max\{n, m, p\}$. We make a mild assumption that $N \geq \tau$, which can be easily satisfied by increasing N .

1) Packing: A packing algorithm that converts a vector of integers in $\mathbb{Z}_q$ into a polynomial in $\mathcal{R}_q$ is presented.

1) $\text{Pack}_k : \mathbb{Z}_q^k \rightarrow \mathcal{R}_q$. For $k \in \mathbb{N}$ such that $k \leq \tau$, the packing of a vector $a = [a_0; a_1; \dots; a_{k-1}] \in \mathbb{Z}_q^k$ is defined by

$$\text{Pack}_k(a) := a_0 + a_1 X^{N/\tau} + \dots + a_{k-1} X^{(k-1)N/\tau} \in \mathcal{R}_q.$$

The algorithm Pack_k assigns the elements of a vector as the coefficients of the terms $X^0, X^{N/\tau}, \dots, X^{(\tau-1)N/\tau}$ in sequence. These terms will be referred to as the *packing slots*.

A matrix–vector multiplication can be expressed with polynomial additions and multiplications using the packing algorithm. For $K \in \mathbb{Z}_q^{h \times l}$ and $z \in \mathbb{Z}_q^l$ such that $h \leq \tau$ and $l \leq \tau$, consider the matrix–vector multiplication

$$Kz = \sum_{i=0}^{l-1} K_i \cdot z_i \in \mathbb{Z}_q^h \quad (17)$$

where K_i denotes the $(i+1)$ th column of K and z_i denotes the $(i+1)$ th element of z . Then, it holds that

$$\text{Pack}_h(Kz) = \sum_{i=0}^{l-1} \text{Pack}_h(K_i) \cdot z_i. \quad (18)$$

In fact, the right-hand side can be evaluated homomorphically with l external products and $l-1$ homomorphic additions, as

$$\sum_{i=0}^{l-1} \text{Enc}'(\text{Pack}_h(K_i)) \boxtimes \text{Enc}(z_i). \quad (19)$$

2) Unpacking Plaintexts (Polynomials): An unpacking algorithm that converts a polynomial in $\mathcal{R}_q$ into a vector of integers in $\mathbb{Z}_q$ is presented. This can be employed to restore the vector Kz from $\text{Pack}_h(Kz)$ in (18).

1) $\text{UnpackPt}_k : \mathcal{R}_q \rightarrow \mathbb{Z}_q^k$. For $k \in \mathbb{N}$ such that $k \leq \tau$, the unpacking of a polynomial $m = \sum_{i=0}^{N-1} m_i X^i \in \mathcal{R}_q$ is defined by

$$\text{UnpackPt}_k(m) := [m_0; m_{N/\tau}; \dots; m_{(k-1)N/\tau}] \in \mathbb{Z}_q^k.$$

The unpacking algorithm UnpackPt_k constructs a k -dimensional vector from the first k packing slots of a given polynomial. Hence, applying UnpackPt_h to (18) yields

$$Kz = \text{UnpackPt}_h(\sum_{i=0}^{l-1} \text{Pack}_h(K_i) \cdot z_i).$$

This implies that Kz can be evaluated homomorphically using (19) if UnpackPt_h can be evaluated homomorphically. By this motivation, we delve deeper into the unpacking algorithm.

Consider a polynomial $m(X) \in \mathcal{R}_q$ written by

$$m(X) := m_0 + m_1 X^{N/\tau} + \dots + m_{\tau-1} X^{(\tau-1)N/\tau}. \quad (20)$$

Since X^N is regarded as -1 in $\mathcal{R}_q$, it follows that:

$$m(X^{\tau+1}) = m_0 - m_1 X^{N/\tau} + \dots - m_{\tau-1} X^{(\tau-1)N/\tau}.$$

Therefore, by adding and subtracting $m(X^{\tau+1})$ from $m(X)$, we can separate the terms, as

$$\begin{aligned} m_E(X) &:= \frac{-q+1}{2} \cdot m(X) + \frac{-q+1}{2} \cdot m(X^{\tau+1}) \\ &= m_0 + m_2 X^{2N/\tau} + \dots + m_{\tau-2} X^{(\tau-2)N/\tau} \\ m_O(X) &:= \left(\frac{-q+1}{2} \cdot m(X) - \frac{-q+1}{2} \cdot m(X^{\tau+1}) \right) \cdot X^{-N/\tau} \\ &= m_1 + m_3 X^{2N/\tau} + \dots + m_{\tau-1} X^{(\tau-2)N/\tau} \end{aligned}$$

where the scalar $(-q+1)/2$, which is the multiplicative inverse² of $2 \in \mathbb{Z}_q$, is premultiplied to halve the doubled coefficients. The monomial $X^{-N/\tau}$ is multiplied when obtaining $m_O(X)$ to simply match the forms of $m_E(X)$ and $m_O(X)$.

By repeating this process, each coefficient of $m(X)$ can be extracted, thus obtaining $\text{UnpackPt}_k(m(X))$. The entire process is summarized in Algorithm 1 and also depicted in Fig. 1. The operation BitReverse in Algorithm 1 reverses the $(\log_2 \tau)$ -digit binary expression of the input. For example, if $\tau = 8$, then $\text{BitReverse}(\tau/2) = \text{BitReverse}(4) = 1$ because the three-digit binary expression of 4 is $[100]_2$, and its reverse $[001]_2$ is 1 in the decimal expression. This is necessary to rearrange the indices of the extracted coefficients in the sequential order (see Fig. 1).

3) Unpacking Ciphertexts: We now present a method to evaluate UnpackPt_k homomorphically over encrypted data. To this end, we introduce the following definitions [10].

1) (Polynomial automorphism) $\Psi_\theta : \mathcal{R}_q \rightarrow \mathcal{R}_q$. For an odd $\theta \in \mathbb{N}$, the automorphism of a polynomial $m(X) \in \mathcal{R}_q$ is defined by

$$\Psi_\theta(m(X)) := m(X^\theta) \in \mathcal{R}_q.$$

2) (Automorphism key) For an odd $\theta \in \mathbb{N}$, the automorphism key $\text{ak}_\theta \in \mathcal{R}_q^{2 \times 2d}$ for the automorphism Ψ_θ under the secret key sk is defined by $\text{ak}_\theta := \text{Enc}'(\Psi_\theta(\text{sk}))$.

3) (Ring-LWE ciphertext automorphism) $\Phi_\theta : \mathcal{R}_q^2 \times \mathcal{R}_q^{2 \times 2d} \rightarrow \mathcal{R}_q^2$. For an odd $\theta \in \mathbb{N}$, the automorphism of $\mathbf{c} = [b; a] \in \mathcal{R}_q^2$ is defined by

$$\Phi_\theta(\mathbf{c}, \text{ak}_\theta) := [\Psi_\theta(b); 0] - \text{ak}_\theta \boxtimes [\Psi_\theta(a); 0] \text{ mod } q \in \mathcal{R}_q^2.$$

The following proposition states the homomorphic property of the ciphertext automorphism Φ_θ .

Proposition 3: For any $\mathbf{c} = [b; a] \in \mathcal{R}_q^2$ and an odd $\theta \in \mathbb{N}$

$$\text{Dec}(\Phi_\theta(\mathbf{c}, \text{ak}_\theta)) = \Psi_\theta(\text{Dec}(\mathbf{c})) + \Delta \text{ mod } q$$

for some $\Delta \in \mathcal{R}_q$ such that $\|\Delta\| \leq \sigma_{\text{Mult}}$. $\square$

²Observe that $2 \cdot (-q+1)/2 \text{ mod } q = -q+1 \text{ mod } q = 1$.

Algorithm 1: Unpacking Plaintexts.

Input: $m(X) \in \mathcal{R}_q$
Procedure $\text{UnpackPt}_k(m(X))$

```

1:  $m_0 \leftarrow m$ 
2: for  $(\zeta = \tau; \zeta > 1; \zeta = \zeta/2)$  do
3:   for  $(\omega = 0; \omega < \tau; \omega = \omega + \zeta)$  do
4:      $m_\omega \leftarrow (-q + 1)/2 \cdot m_\omega$ 
5:      $\text{tmp}_\omega \leftarrow m_\omega(X^{\zeta+1})$ 
6:      $m_{\omega+\zeta/2} \leftarrow (m_\omega - \text{tmp}_\omega) \cdot X^{-N/\zeta}$ 
7:      $m_\omega \leftarrow m_\omega + \text{tmp}_\omega$ 
8:   end for
9: end for
10: for  $(\text{idx} = 0; \text{idx} < \tau; \text{idx} = \text{idx} + 1)$  do
11:    $\text{BRidx} \leftarrow \text{BitReverse}(\text{idx})$ 
12:   if  $(\text{idx} < \text{BRidx})$  then
13:      $m_{\text{idx}}, m_{\text{BRidx}} \leftarrow m_{\text{BRidx}}, m_{\text{idx}}$ 
14:   end if
15: end for
16: return  $[m_0; \dots; m_{k-1}] \in \mathbb{Z}_q^k$ 

```

Proof: See Appendix C. ■

Making use of Proposition 3, we define an algorithm $\text{UnpackCt}_k: \mathcal{R}_q^2 \rightarrow \mathcal{R}_q^k$ that homomorphically evaluates UnpackPt_k through Algorithm 2. Observe that polynomial addition and automorphism Ψ_θ have been substituted by $\boxplus$ and Φ_θ , respectively, and that the set of automorphism keys ak_θ is necessary to execute UnpackCt_k .

The rest of the section studies the growth of errors caused by the packing and unpacking algorithms. Note that the external product $\boxtimes$ is executed $(\log_2 \tau)$ -times (one for each Φ_θ) in obtaining each $\mathbf{c}_i$ of Algorithm 2. This accounts for the bound on Δ stated in the following lemma.

Lemma 2: For $\mathbf{c} \in \mathcal{R}_q^2$ and $k \in \mathbb{N}$ such that $k \leq \tau$, let $\text{UnpackCt}_k(\mathbf{c}) = [\mathbf{c}_0; \dots; \mathbf{c}_{k-1}]$ and $\text{Dec}(\mathbf{c}) = \sum_{j=0}^{N-1} c_j X^j$. Then, for each $i = 0, \dots, k-1$,

$$\text{UnpackPt}_k(\text{Dec}(\mathbf{c}_i)) = \begin{bmatrix} c_{iN/\tau} \\ 0_{(k-1) \times 1} \end{bmatrix} + \Delta \bmod q \quad (21)$$

for some $\Delta \in \mathbb{Z}_q^k$ such that $\|\Delta\| \leq \log_2 \tau \cdot \sigma_{\text{Mult}}$. □

Proof: See Appendix D. ■

The implication of Lemma 2 is that UnpackCt_k distributes the coefficient in the i -th packing slot of $\text{Dec}(\mathbf{c})$ into the constant term of $\text{Dec}(\mathbf{c}_i)$, while introducing additional errors (Δ in (21)) in the packing slots of $\text{Dec}(\mathbf{c}_i)$. As in Proposition 2, the newborn error Δ is guaranteed to have a constant bound regardless of the errors accumulated in $\mathbf{c}$.

The following proposition extends the result of Lemma 2 and presents a bound on the growth of errors when homomorphically evaluating the matrix–vector multiplication (17). This will be extensively used to analyze the performance of the encrypted controller proposed in the next subsection.

Proposition 4: For $\mathbf{c} \in \mathcal{R}_q^2$ and $K \in \mathbb{Z}_q^{h \times l}$, where $h \leq \tau$ and $l \leq \tau$, let K_i denote the $(i+1)$ -th column of K for $i =$

Algorithm 2: Unpacking Ciphertexts

Input: $\mathbf{c} \in \mathcal{R}_q^2$ and ak_θ for $\theta \in \{2^\xi + 1 \mid 2 \leq 2^\xi \leq \tau, \xi \in \mathbb{N}\}$
Procedure $\text{UnpackCt}_k(\mathbf{c})$

```

1:  $\mathbf{c}_0 \leftarrow \mathbf{c}$ 
2: for  $(\zeta = \tau; \zeta > 1; \zeta = \zeta/2)$  do
3:   for  $(\omega = 0; \omega < \tau; \omega = \omega + \zeta)$  do
4:      $\mathbf{c}_\omega \leftarrow (-q + 1)/2 \cdot \mathbf{c}_\omega$ 
5:      $\text{tmp}_\omega \leftarrow \Phi_{\zeta+1}(\mathbf{c}_\omega, \text{ak}_{\zeta+1})$ 
6:      $\mathbf{c}_{\omega+\zeta/2} \leftarrow (\mathbf{c}_\omega \boxplus (-\text{tmp}_\omega)) \cdot X^{-N/\zeta}$ 
7:      $\mathbf{c}_\omega \leftarrow \mathbf{c}_\omega \boxplus \text{tmp}_\omega$ 
8:   end for
9: end for
10: for  $(\text{idx} = 0; \text{idx} < \tau; \text{idx} = \text{idx} + 1)$  do
11:    $\text{BRidx} \leftarrow \text{BitReverse}(\text{idx})$ 
12:   if  $(\text{idx} < \text{BRidx})$  then
13:      $\mathbf{c}_{\text{idx}}, \mathbf{c}_{\text{BRidx}} \leftarrow \mathbf{c}_{\text{BRidx}}, \mathbf{c}_{\text{idx}}$ 
14:   end if
15: end for
16: return  $[\mathbf{c}_0; \dots; \mathbf{c}_{k-1}] \in \mathcal{R}_q^{2k}$ 

```

$0, 1, \dots, l-1$, and $[\mathbf{c}_0; \dots; \mathbf{c}_{l-1}] := \text{UnpackCt}_l(\mathbf{c})$. Then

$$\begin{aligned} & \text{UnpackPt}_h(\text{Dec}(\sum_{i=0}^{l-1} \text{Enc}'(\text{Pack}_h(K_i)) \boxtimes \mathbf{c}_i)) \\ &= K \cdot \text{UnpackPt}_l(\text{Dec}(\mathbf{c})) + \Delta \bmod q \end{aligned} \quad (22)$$

for some $\Delta \in \mathbb{Z}_q^h$ such that $\|\Delta\| \leq l(1 + \|K^\top\| \log_2 \tau) \cdot \sigma_{\text{Mult}}$. □

Proof: See Appendix E. ■

B. Encrypted Controller Design

We propose an encrypted controller that integrates the introduced packing and unpacking algorithms, thus requiring fewer homomorphic operations at each time step compared to (8). With a slight abuse of notation, we denote the state, input, output, and initial state of the encrypted controller by $\mathbf{x}(t)$, $\mathbf{y}(t)$, $\mathbf{u}(t)$, and $\mathbf{x}^{\text{ini}}$, respectively, as done in (8).

During the offline procedure, the automorphism keys ak_θ for $\theta \in \{2^\xi + 1 \mid 2 \leq 2^\xi \leq \tau, \xi \in \mathbb{N}\}$ are generated, which are necessary to execute Algorithm 2. Then, the matrices F , G , and H and the vector $\mathbf{x}^{\text{ini}}$ of (5) are encrypted as follows; let us denote the $(i+1)$ -th column of F , G , and H by F_i , G_i , and H_i , respectively. Each column of F , G , and H is quantized, packed, and encrypted as

$$\mathbf{F}_i := \text{Enc}'(\text{Pack}_n(F_i)) \in \mathcal{R}_q^{2 \times 2d}, \quad i = 0, \dots, n-1$$

$$\mathbf{G}_i := \text{Enc}'(\text{Pack}_n(G_i/s)) \in \mathcal{R}_q^{2 \times 2d}, \quad i = 0, \dots, p-1$$

$$\mathbf{H}_i := \text{Enc}'(\text{Pack}_m(H_i/s)) \in \mathcal{R}_q^{2 \times 2d}, \quad i = 0, \dots, n-1$$

and the vector $\mathbf{x}^{\text{ini}}$ is quantized, packed, and encrypted as

$$\mathbf{x}^{\text{ini}} := \text{Enc}(\text{Pack}_n(\lceil \mathbf{x}^{\text{ini}}/r \rceil / (sL))) \in \mathcal{R}_q^2$$

using the quantization step size r , and the scale factors s and L in (7).

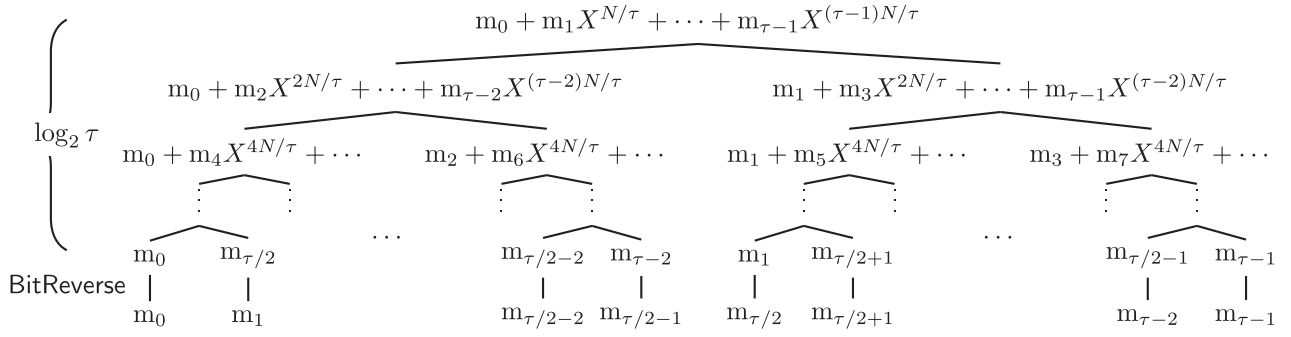


Fig. 1. Illustration of $\text{UnpackPt}_k(m)$ for $m(X)$ given in (20).

At each time step $t \geq 0$ of the online procedure, the sensor quantizes, packs, and encrypts the plant output $y(t)$, as

$$\mathbf{y}(t) := \text{Enc}(\text{Pack}_p(\lceil y(t)/r \rceil / L)) \in \mathcal{R}_q^2.$$

It is then sent to the encrypted controller, designed as

$$\mathbf{x}(t+1) = \left(\sum_{i=0}^{n-1} \mathbf{F}_i \boxtimes \mathbf{x}_i(t) \right) \boxplus \left(\sum_{i=0}^{p-1} \mathbf{G}_i \boxtimes \mathbf{y}_i(t) \right) \quad (23a)$$

$$\mathbf{u}(t) = \sum_{i=0}^{n-1} \mathbf{H}_i \boxtimes \mathbf{x}_i(t) \quad (23b)$$

$$\mathbf{x}(0) = \mathbf{x}^{\text{ini}} \quad (23c)$$

where the summations are taken with respect to $\boxplus$, and

$$\begin{aligned} [\mathbf{x}_0(t); \dots; \mathbf{x}_{n-1}(t)] &:= \text{UnpackCt}_n(\mathbf{x}(t)) \\ [\mathbf{y}_0(t); \dots; \mathbf{y}_{p-1}(t)] &:= \text{UnpackCt}_p(\mathbf{y}(t)). \end{aligned} \quad (23d)$$

The output $\mathbf{u}(t)$ is transmitted to the actuator, where it is decrypted, unpacked, and then scaled down to obtain the plant input $u(t)$, as

$$u(t) = rs^2L \cdot \text{UnpackPt}_m(\text{Dec}(\mathbf{u}(t))). \quad (23e)$$

C. Performance Analysis

As done in Section III, we construct a controller over $\mathbb{R}$ whose performance is equivalent to that of the encrypted controller (23). Let us abuse notation and define $u_0(t)$ and $x_0(t)$ (distinct from (9)), by

$$u_0(t) := rs^2L \cdot \text{UnpackPt}_m(\text{Dec}(\mathbf{u}(t))) \in \mathbb{R}^m$$

$$x_0(t) := rsL \cdot \text{UnpackPt}_n(\text{Dec}(\mathbf{x}(t))) \in \mathbb{R}^n.$$

Then, we define the perturbations $e_0^x(t)$, $e_0^u(t)$, and e_0^{ini} by

$$\begin{aligned} e_0^x(t) &:= x_0(t+1) - (Fx_0(t) + Gy(t)), \quad e_0^{\text{ini}} := x_0(0) - x^{\text{ini}} \\ e_0^u(t) &:= u_0(t) - Hx_0(t). \end{aligned}$$

It is easily verified that the performance of (23) is equivalent to that of the following controller over $\mathbb{R}$ with $u(t) = u_0(t)$:

$$\begin{aligned} x_0(t+1) &= Fx_0(t) + Gy(t) + e_0^x(t), \quad x_0(0) = x^{\text{ini}} + e_0^{\text{ini}} \\ u_0(t) &= Hx_0(t) + e_0^u(t). \end{aligned} \quad (24)$$

The following lemma states that the perturbations of (24) remain bounded for all $t \geq 0$ if the modulus q is chosen sufficiently

large. To state the lemma, we slightly adapt α , β , and $\bar{\eta}$ defined in Lemma 1 and define

$$\alpha'(r, s, L) := \alpha(r, s, L) + rsL \left(n \|F^\top\| + \frac{p \|G^\top\|}{s} \right) \log_2 \tau \cdot \sigma_{\text{Mult}}$$

$$\beta'(r, s, L) := \beta(r, s, L) + rsLn \|H^\top\| \log_2 \tau \cdot \sigma_{\text{Mult}}$$

$$\bar{\eta}'(r, s, L) := \eta(\alpha'(r, s, L), \beta'(r, s, L), \gamma(r, s, L)).$$

Lemma 3: Given the parameters $r > 0$, $1/s \in \mathbb{N}$, and $1/L \in \mathbb{N}$, the perturbations of (24) remain bounded as

$$\|e_0^x(t)\| \leq \alpha'(r, s, L), \quad \|e_0^u(t)\| \leq \beta'(r, s, L), \quad \|e_0^{\text{ini}}\| \leq \gamma(r, s, L)$$

for all $t \geq 0$ if

$$q > 2 \max \left\{ \frac{\bar{\eta}'(r, s, L)}{rsL}, \frac{\|H\| \bar{\eta}'(r, s, L) + \beta'(r, s, L)}{rs^2L} \right\}. \quad (25)$$

Proof: See Appendix F. ■

Note that Lemma 3 is a generalization of Lemma 1 because $\alpha' = \alpha$ and $\beta' = \beta$, and $\bar{\eta}' = \bar{\eta}$ when $\tau = 1$, in which the constant term is the only packing slot. It is immediate from Lemma 3 that the performance error in (6) can be made arbitrarily small with the choice of parameters $\{q, r, s, L\}$.

Theorem 2: For given $\epsilon > 0$, if the parameters $r > 0$, $1/s \in \mathbb{N}$, and $1/L \in \mathbb{N}$ satisfy

$$\beta'(r, s, L) \leq \frac{\epsilon}{2} \quad \text{and} \quad \bar{\eta}'(r, s, L) \leq M \left\| \begin{bmatrix} x_p^{\text{ini}} \\ x^{\text{ini}} \end{bmatrix} \right\| + \frac{\epsilon}{2 \|H\|} \quad (26)$$

and the modulus q satisfies (25), then the encrypted controller (23) guarantees that (6) holds for all $t \geq 0$. ■

Sketch of Proof: The proof is similar to that of Theorem 1. Simply replace the controller (11) and the functions α , β , and $\bar{\eta}$ in the proof of Theorem 1 with the controller (24) and the functions α' , β' , and $\bar{\eta}'$, respectively. ■

Since $\alpha'(r, s, L)$ and $\beta'(r, s, L)$ vanish at the origin as with $\gamma(r, s, L)$, the condition (26) can be easily satisfied by choosing sufficiently small r , s , and L .

Remark 3: We provide some comparisons of the encrypted controllers (8) and (23) in terms of computation load and memory efficiency. Table I compares the number of homomorphic operations performed by the encrypted controllers (8) and (23) at each time step. It should be noted that UnpackCt_k takes

TABLE I

NUMBER OF HOMOMORPHIC OPERATIONS PERFORMED AT EACH TIME STEP

Method	(8)	(23)
Enc	p	1
Dec	m	1
$\boxplus$	$n^2 + n(p + m - 1) - m$	$2n + p - 2$
$\boxminus$	$n^2 + n(p + m)$	$2n + p$
UnpackCt _k	-	2

$(\tau - 1)$ -external products (one for each automorphism Φ_{i+1}) for execution, and that the time complexity of the external product is dominant over other operations such as $\boxplus$ or BitReverse. Therefore, (23) is expected to take less computation time when $n^2 + n(p + m) > 2n + p + 2\tau - 2$, which, for example, holds for all $n \geq 2$ if $m = p = 1$. This is well supported by the simulation results (see Section V). In terms of memory usage, one needs to store $(n^2 + np + nm)$ -Ring-GSW ciphertexts for the implementation of (8), which correspond to the encryptions of each component of the matrices F , G , and H . In this respect, (23) is more memory efficient, as it packs each column of the matrices prior to encryption. Consequently, it requires storing only $(2n + p + \log_2 \tau)$ -Ring-GSW ciphertexts, where the term $\log_2 \tau$ accounts for the automorphism keys. $\square$

V. SIMULATION RESULTS

Simulation results of the proposed methods applied to the four-tank system [23] are provided. The model of the form (4) is obtained as

$$A = \begin{bmatrix} 0.9984 & 0 & 0.0042 & 0 \\ 0 & 0.9989 & 0 & -0.0033 \\ 0 & 0 & 0.9958 & 0 \\ 0 & 0 & 0 & 0.9967 \end{bmatrix},$$

$$B = \begin{bmatrix} 0.0083 & 0 \\ 0 & 0.0063 \\ 0 & 0.0048 \\ 0.0031 & 0 \end{bmatrix} \quad C = \begin{bmatrix} 0.5 & 0 & 0 & 0 \\ 0 & 0.5 & 0 & 0 \end{bmatrix}$$

by discretizing the system from [23] with the sampling time of 100 ms. An observer-based controller that stabilizes the plant is designed as

$$z(t+1) = (A + BK - LC)z(t) + Ly(t), \quad u(t) = Kz(t) \quad (27)$$

where the gains $K \in \mathbb{R}^{2 \times 4}$ and $L \in \mathbb{R}^{4 \times 2}$ are given by

$$K = \begin{bmatrix} -0.7905 & 0.1579 & -0.2745 & -0.2686 \\ -0.1552 & -0.7874 & -0.3427 & 0.3137 \end{bmatrix}$$

$$L = \begin{bmatrix} 0.7815 & 0 & 0.3190 & 0 \\ 0 & 0.7816 & 0 & -0.3199 \end{bmatrix}^\top.$$

Next, the state matrix of (27) is converted to an integer matrix. For the sake of simplicity, we use the method of [8, Lemma 1], which utilizes re-encryption; let the state dynamics of (27) be

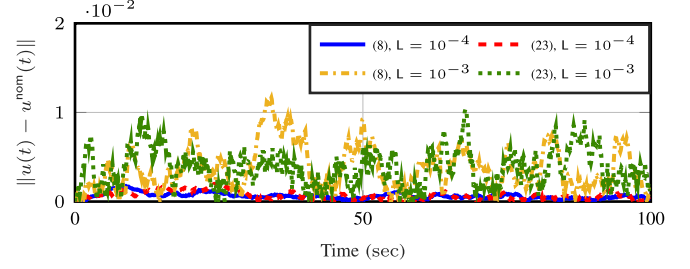


Fig. 2. Performance error $\|u(t) - u^{\text{nom}}(t)\|$ of the encrypted controllers (8) and (23) for different values of L .

rewritten as

$$z(t+1) = (A + BK - LC - RK)z(t) + \begin{bmatrix} L & R \end{bmatrix} \begin{bmatrix} y(t) \\ u(t) \end{bmatrix}$$

where $u(t)$ is regarded as a fed-back input, and the matrix $R \in \mathbb{R}^{4 \times 2}$ is designed as

$$R = \begin{bmatrix} -1.6879 & 0.4148 & 0.2880 & -0.7385 \\ -0.6892 & -2.1054 & 4.1931 & -2.0807 \end{bmatrix}^\top$$

so that $\det(\lambda I_4 - (A + BK - LC - RK)) = \lambda(\lambda - 1)(\lambda + 1)(\lambda - 2)$. Then, a coordinate transformation $x(t) = Tz(t)$ yields a controller of the form (5) with

$$F = T(A + BK - LC - RK)T^{-1} \in \mathbb{Z}^{4 \times 4}$$

$$G = T \begin{bmatrix} L & R \end{bmatrix}, \quad H = KT^{-1}$$

which is in the modal canonical form. The initial values are chosen as $x_p^{\text{ini}} = [1; 1; 1; 1]$ and $x^{\text{ini}} = [-1; 0; -1; 0]$.

The performance of the encrypted controllers (8) and (23) are compared in terms of computation time and performance error. For the simulation: 1) the error distribution ψ is set as the discrete Gaussian distribution with standard deviation 3.2, and bounded by $\sigma = 19.2$; 2) the modulus q , the special modulus P , and the decomposition base ν of the Ring-LWE-based scheme are set as $q \approx 2^{56}$ such that $1 = q \bmod 2N$, $P \approx 2^{51}$, and $\nu = q$ so that $d = 1$; and 3) the parameters r and s are set as $r = s = 10^{-4}$. We developed our code with Lattigo version 6.1.0 [13], and all experiments are performed on a laptop with Intel Core Ultra 5 125H CPU running 18 threads at 1.20 GHz. The simulation code for this section is fully available online.³

Fig. 2 depicts the performance error $\|u(t) - u^{\text{nom}}(t)\|$ of the encrypted controllers for different values of L , when $N = 2^{12}$. It is observed that the performance error remains bounded in all cases, which asserts that the proposed encrypted controllers can operate for an infinite-time horizon without bootstrapping.⁴ Moreover, decreasing the parameter L is observed to reduce the effects of quantization and error growth, leading to better performance.

³[Online]. Available: <https://github.com/CDSL-EncryptedControl/CDSL>

⁴The selected sets of parameters $\{q, r, s, L\}$ satisfy the conditions (15) and (25). For example, the lower bounds of ϵ such that (16) and (26) each hold are computed as 0.0275 and 177.9048, respectively, for the case of $L = 10^{-4}$. Nevertheless, the performance error in the simulation remains below 0.005 for 100 s (1000 iterations) in these cases.

TABLE II
COMPUTATION TIME AT EACH TIME STEP

N	Method	Mean (ms)	Max (ms)	Min (ms)	Std
2^{12}	(8)	17.77	18.47	17.07	0.70
	(23)	8.46	8.53	8.39	0.07
2^{13}	(8)	38.06	41.07	35.05	3.01
	(23)	16.85	17.33	16.36	0.48

*Number of homomorphic operations performed at each time step are specified in Table I ($n = 4$ and $m = p = 2$).

Table II shows the mean, maximum, minimum, and standard deviation of the elapsed computation time from the calculation of sensor outputs to the generation of actuator inputs at each time step when $N = 2^{12}$ and $N = 2^{13}$, which ensure 128-bit and 256-bit security [17], respectively. Here, the parameter L is fixed as $L = 10^{-4}$. It can be seen that the packing algorithm notably improves the computational efficiency, and that there exists a tradeoff between computation time and security level. In all cases, the computation time falls within the sampling time, suggesting the practical applicability of the proposed methods for real-time control systems.

VI. CONCLUSION

In this article, we have introduced an encrypted dynamic controller that executes an unlimited number of recursive homomorphic multiplications on a Ring-LWE-based cryptosystem without bootstrapping. By deriving a controller over $\mathbb{R}$ whose performance is equivalent to that of the encrypted controller, we have shown that the effect of error growth can be arbitrarily bounded by means of closed-loop stability. We have established a sufficient condition on the parameters $\{q, r, s, L\}$, under which the proposed encrypted controller guarantees desired control performance. Furthermore, we have proposed and applied a novel packing algorithm that leads to improved computational and memory efficiency.

APPENDIX A PROOF OF LEMMA 1

We start by specifying the perturbations $e_0^x(t)$, $e_0^u(t)$, and e_0^{ini} . It follows from Proposition 1 that:

$$\begin{aligned} \frac{G}{s} \text{Dec}(y(t)) \bmod q &= \frac{G}{s} \left(\frac{1}{L} \left\lceil \frac{y(t)}{r} \right\rceil + \Delta(t) \right) \bmod q \\ &= \frac{Gy(t)}{rsL} + \frac{G}{sL} \left(\left\lceil \frac{y(t)}{r} \right\rceil - \frac{y(t)}{r} \right) + \frac{G}{s} \Delta(t) \bmod q \end{aligned}$$

for some $\Delta(t) \in \mathcal{R}_q^p$ bounded as $\|\Delta(t)\| \leq \sigma$. Then, the following are immediate from Proposition 1 and (3):

$$\begin{aligned} \frac{x(t+1)}{rsL} &= \text{Dec}((F \boxtimes x(t)) \boxplus (G \boxtimes y(t))) \\ &= \frac{Fx(t) + Gy(t)}{rsL} + \Delta^x(t) \bmod q \\ \frac{u(t)}{rs^2L} &= \text{Dec}(H \boxtimes x(t)) = \frac{Hx(t)}{rs^2L} + \Delta^u(t) \bmod q \\ \frac{x(0)}{rsL} &= \text{Dec}(x^{ini}) = \frac{x^{ini}}{rsL} + \Delta^{ini} \bmod q \end{aligned} \quad (28)$$

for some $\Delta^x(t) \in \mathcal{R}_q^n$, $\Delta^u(t) \in \mathcal{R}_q^m$, and $\Delta^{ini} \in \mathcal{R}_q^n$ bounded as

$$\begin{aligned} \|\Delta^x(t)\| &\leq (n+p)\sigma_{\text{Mult}} + \frac{\|G\|}{2sL} + \frac{\|G\|}{s}\sigma = \frac{\alpha(r, s, L)}{rsL} \\ \|\Delta^u(t)\| &\leq n\sigma_{\text{Mult}} = \frac{\beta(r, s, L)}{rs^2L} \\ \|\Delta^{ini}\| &\leq \sigma + \frac{1}{2sL} = \frac{\gamma(r, s, L)}{rsL} \end{aligned} \quad (29)$$

for all $t \geq 0$.

As with (9), let $\Delta_i^x(t) \in \mathbb{Z}_q^n$, $\Delta_i^u(t) \in \mathbb{Z}_q^m$, and $\Delta_i^{ini} \in \mathbb{Z}_q^n$ denote the vectors consisting of the i -th coefficients of $\Delta^x(t)$, $\Delta^u(t)$, and Δ^{ini} , respectively. Then, using (10) and (28), the perturbations $e_0^x(t)$, $e_0^u(t)$, and e_0^{ini} can be expressed as

$$\begin{aligned} e_0^x(t) &= rsL \cdot \left(\frac{Fx_0(t) + Gy(t)}{rsL} + \Delta_0^x(t) \bmod q \right) \\ &\quad - (Fx_0(t) + Gy(t)) \\ e_0^u(t) &= rs^2L \cdot \left(\frac{Hx_0(t)}{rs^2L} + \Delta_0^u(t) \bmod q \right) - Hx_0(t) \\ e_0^{ini} &= rsL \cdot \left(\frac{x^{ini}}{rsL} + \Delta_0^{ini} \bmod q \right) - x^{ini}. \end{aligned} \quad (30)$$

We now show that the modulo operations in (30) can be omitted. Let us consider an auxiliary dynamics written by

$$\begin{aligned} z(t+1) &= Fz(t) + Gy(t) + rsL \cdot \Delta_0^x(t) \\ v(t) &= Hz(t) + rs^2L \cdot \Delta_0^u(t) \\ z(0) &= x^{ini} + rsL \cdot \Delta_0^{ini} \end{aligned} \quad (31)$$

where $z(t) \in \mathbb{R}^n$ and $v(t) \in \mathbb{R}^m$. Here, $y(t)$ is the plant output of the closed-loop system (4) with (31), where the input $u(t)$ of (4) is replaced with $v(t)$. We claim that

$$\|z(t)\| < rsL \frac{q}{2} \quad \text{and} \quad \|v(t)\| < rs^2L \frac{q}{2} \quad (32)$$

hold for all $t \geq 0$. With $\chi(t) := [x_p(t); z(t)]$, the closed-loop system (4) with (31) can be written by

$$\chi(t+1) = \bar{A}\chi(t) + \begin{bmatrix} rs^2L \cdot B\Delta_0^u(t) \\ rsL \cdot \Delta_0^x(t) \end{bmatrix}, \quad \chi(0) = \begin{bmatrix} x_p^{ini} \\ x^{ini} + rsL \cdot \Delta_0^{ini} \end{bmatrix}.$$

It follows from (29) and the definitions of M and λ that:

$$\begin{aligned} \|\chi(t)\| &\leq \|\chi(0)\| \\ &= \left\| \bar{A}^t \chi(0) + \sum_{k=0}^{t-1} \bar{A}^k \begin{bmatrix} rs^2L \cdot B\Delta_0^u(t-1-k) \\ rsL \cdot \Delta_0^x(t-1-k) \end{bmatrix} \right\| \\ &\leq \|\bar{A}^t\| \cdot \|\chi(0)\| + (\|B\|\beta(r, s, L) + \alpha(r, s, L)) \sum_{k=0}^{t-1} \|\bar{A}^k\| \\ &\leq M \left(\left\| \begin{bmatrix} x_p^{ini} \\ x^{ini} \end{bmatrix} \right\| + \gamma(r, s, L) + \frac{\|B\|\beta(r, s, L) + \alpha(r, s, L)}{1 - \lambda} \right) \\ &= \bar{\eta}(r, s, L) \\ \|v(t)\| &\leq \|H\| \cdot \|z(t)\| + rs^2L \|\Delta_0^u(t)\| \\ &\leq \|H\| \bar{\eta}(r, s, L) + \beta(r, s, L). \end{aligned} \quad (33)$$

Hence, the claim (32) holds by (15) and (33).

Given that the claim (32) holds, we can rewrite (31) as

$$\begin{aligned} z(t+1) &= \text{rsL} \cdot \left(\frac{Fz(t) + Gy(t)}{\text{rsL}} + \Delta_0^x(t) \bmod q \right) \\ v(t) &= \text{rs}^2\text{L} \cdot \left(\frac{Hz(t)}{\text{rs}^2\text{L}} + \Delta_0^u(t) \bmod q \right) \\ z(0) &= \text{rsL} \cdot \left(\frac{x_0^{\text{ini}}}{\text{rsL}} + \Delta_0^{\text{ini}} \bmod q \right) \end{aligned}$$

because $a = a \bmod q$ for all $a \in \mathbb{Z}$ such that $|a| < q/2$. Plugging (30) into (11) leads to $x_0(t) \equiv z(t)$ and $u_0(t) \equiv v(t)$. Therefore, the perturbations can be further specified as

$$e_0^x(t) = \text{rsL} \cdot \Delta_0^x(t), \quad e_0^u(t) = \text{rs}^2\text{L} \cdot \Delta_0^u(t), \quad e_0^{\text{ini}} = \text{rsL} \cdot \Delta_0^{\text{ini}}$$

and this concludes the proof combined with (29).

APPENDIX B PROOF OF THEOREM 1

Let us denote the states of the plant and the controller of the closed-loop system (4) with (5) by $x_p^{\text{nom}}(t)$ and $x^{\text{nom}}(t)$, respectively. Considering (13), we define the error variable, as

$$\chi_e(t) := \begin{bmatrix} x_p(t) - x_p^{\text{nom}}(t) \\ x_0(t) - x^{\text{nom}}(t) \end{bmatrix}$$

which leads us to the error dynamics

$$\chi_e(t+1) = \bar{A}\chi_e(t) + \begin{bmatrix} B & 0 \\ 0 & I_n \end{bmatrix} \begin{bmatrix} e_0^u(t) \\ e_0^x(t) \end{bmatrix}, \quad \chi_e(0) = \begin{bmatrix} 0 \\ e_0^{\text{ini}} \end{bmatrix}.$$

Since $\|u(t) - u^{\text{nom}}(t)\| = \|H(x_0(t) - x^{\text{nom}}(t)) + e_0^u(t)\|$, it suffices to show that $\chi_e(t)$ and $e_0^u(t)$ are bounded as

$$\|\chi_e(t)\| \leq \frac{\epsilon}{2\|H\|} \quad \text{and} \quad \|e_0^u(t)\| \leq \frac{\epsilon}{2} \quad (34)$$

for all $t \geq 0$.

In fact, the second condition of (34) follows from (14) and (16). Regarding the first condition, we can derive a bound on $\|\chi_e(t)\|$ based on the error dynamics as done for (33):

$$\begin{aligned} \|\chi_e(t)\| &\leq \|\bar{A}^t\| \|\chi_e(0)\| + (\|B\| \beta(r, s, L) + \alpha(r, s, L)) \sum_{k=0}^{t-1} \|\bar{A}^k\| \\ &\leq M \left(\gamma(r, s, L) + \frac{(\|B\| \beta(r, s, L) + \alpha(r, s, L))}{1 - \lambda} \right) \\ &= \bar{\eta}(r, s, L) - M \left\| \begin{bmatrix} x_p^{\text{ini}} \\ x^{\text{ini}} \end{bmatrix} \right\| \leq \frac{\epsilon}{2\|H\|} \end{aligned}$$

where the last inequality follows from (16). This concludes the proof.

APPENDIX C PROOF OF PROPOSITION 3

It follows from the definition of Dec and Propositions 1 and 2 that

$$\text{Dec}(\Phi_\theta(\mathbf{c}, \text{ak}_\theta)) = \text{Dec}([\Psi_\theta(\mathbf{b}); 0] - \text{ak}_\theta \boxminus [\Psi_\theta(\mathbf{a}); 0])$$

$$\begin{aligned} &= \Psi_\theta(\mathbf{b}) - \Psi_\theta(\text{sk}) \cdot \text{Dec}([\Psi_\theta(\mathbf{a}); 0]) + \Delta \\ &= \Psi_\theta(\mathbf{b}) - \Psi_\theta(\text{sk}) \cdot \Psi_\theta(\mathbf{a}) + \Delta \end{aligned}$$

for some $\Delta \in \mathcal{R}_q$ bounded as $\|\Delta\| \leq \sigma_{\text{Mult}}$. Since $\Psi_\theta(\mathbf{b}) - \Psi_\theta(\text{sk}) \cdot \Psi_\theta(\mathbf{a}) = \Psi_\theta(\mathbf{b} - \text{sk} \cdot \mathbf{a}) = \Psi_\theta(\text{Dec}(\mathbf{c}))$, this concludes the proof.

APPENDIX D PROOF OF LEMMA 2

For convenience, let us introduce an operation $\text{Slot} : \mathcal{R}_q \rightarrow \mathcal{R}_q$ that takes the packing slots of a given polynomial $m = \sum_{i=0}^{N-1} m_i X^i \in \mathcal{R}_q$ as follows:

$$\text{Slot}(m) := \sum_{i=0}^{\tau-1} m_{iN/\tau} X^{iN/\tau}. \quad (35)$$

For any $m \in \mathcal{R}_q$, $m' \in \mathcal{R}_q$, and odd $\theta \in \mathbb{N}$, it holds that:

- P1) $\text{Slot}(\Psi_\theta(m)) = \Psi_\theta(\text{Slot}(m))$;
- P2) $\text{Slot}(m + m') = \text{Slot}(m) + \text{Slot}(m')$.

P1 holds because N is a power of two and thus is coprime to θ . This implies that the nonpacking slots do not affect the packing slots under the automorphism Ψ_θ . P2 is trivial.

Without loss of generality, we only prove for the case of $i = 0$. For $l = 1, 2, \dots, \log_2 \tau$, let $\mathbf{c}_0^l$ denote $\mathbf{c}_0$ computed at Step 7 of Algorithm 2 when $\zeta = \tau/2^{l-1}$. When $l = 1$ (or $\zeta = \tau$), $\mathbf{c}_0^1$ is obtained as

$$\mathbf{c}_0^1 = \frac{-q+1}{2} \cdot \mathbf{c} \boxplus \Phi_{\tau+1} \left(\frac{-q+1}{2} \cdot \mathbf{c}, \text{ak}_{\tau+1} \right).$$

By Propositions 1 and 3, we have

$$\begin{aligned} \text{Dec}(\mathbf{c}_0^1) &= \frac{-q+1}{2} \cdot \text{Dec}(\mathbf{c}) + \text{Dec} \left(\Phi_{\tau+1} \left(\frac{-q+1}{2} \cdot \mathbf{c}, \text{ak}_{\tau+1} \right) \right) \\ &= \frac{-q+1}{2} \cdot (\text{Dec}(\mathbf{c}) + \Psi_{\tau+1}(\text{Dec}(\mathbf{c}))) + \Lambda_1 \end{aligned}$$

for some $\Lambda_1 \in \mathcal{R}_q$ such that $\|\Lambda_1\| \leq \sigma_{\text{Mult}}$. Then, by P1 and P2, we have

$$\begin{aligned} \text{Slot}(\text{Dec}(\mathbf{c}_0^1)) &= \frac{-q+1}{2} \cdot (\text{Slot}(\text{Dec}(\mathbf{c})) + \text{Slot}(\Psi_{\tau+1}(\text{Dec}(\mathbf{c})))) + \text{Slot}(\Lambda_1) \\ &= \frac{-q+1}{2} \cdot (\text{Slot}(\text{Dec}(\mathbf{c})) + \Psi_{\tau+1}(\text{Slot}(\text{Dec}(\mathbf{c})))) + \text{Slot}(\Lambda_1) \\ &= c_0 + c_{2N/\tau} X^{2N/\tau} + \dots + c_{(\tau-2)N/\tau} X^{(\tau-2)N/\tau} + \text{Slot}(\Lambda_1). \end{aligned} \quad (36)$$

When $l = 2$ (or $\zeta = \tau/2$), $\mathbf{c}_0^2$ is obtained from $\mathbf{c}_0^1$, as

$$\mathbf{c}_0^2 = \frac{-q+1}{2} \cdot \mathbf{c}_0^1 \boxplus \Phi_{\tau/2+1} \left(\frac{-q+1}{2} \cdot \mathbf{c}_0^1, \text{ak}_{\tau/2+1} \right)$$

and analogously to (36), it can be derived that

$$\begin{aligned} \text{Slot}(\text{Dec}(\mathbf{c}_0^2)) &= c_0 + c_{4N/\tau} X^{4N/\tau} + \dots + c_{(\tau-4)N/\tau} X^{(\tau-4)N/\tau} \\ &\quad + \frac{-q+1}{2} \cdot (\text{Slot}(\Lambda_1) + \Psi_{\tau/2+1}(\text{Slot}(\Lambda_1))) + \text{Slot}(\Lambda_2) \end{aligned}$$

for some $\Lambda_2 \in \mathcal{R}_q$ such that $\|\Lambda_2\| \leq \sigma_{\text{Mult}}$. Since $(-q+1)/2$ is the multiplicative inverse of $2 \in \mathbb{Z}_q$ and $\|\Lambda_1\| \leq \sigma_{\text{Mult}}$

$$\left\| \frac{-q+1}{2} \cdot (\text{Slot}(\Lambda_1) + \Psi_{\tau/2+1}(\text{Slot}(\Lambda_1))) + \text{Slot}(\Lambda_2) \right\| \leq 2\sigma_{\text{Mult}}.$$

Applying the same argument repeatedly until $l = \log_2 \tau$ (or $\zeta = 2$), we arrive at

$$\text{Slot}(\text{Dec}(\mathbf{c}_0^{\log_2 \tau})) = \text{Slot}(\text{Dec}(\mathbf{c}_0)) = \mathbf{c}_0 + \text{Slot}(\bar{\Lambda})$$

for some $\bar{\Lambda} \in \mathcal{R}_q$ such that $\|\text{Slot}(\bar{\Lambda})\| \leq \log_2 \tau \cdot \sigma_{\text{Mult}}$, and this concludes the proof.

APPENDIX E PROOF OF PROPOSITION 4

First, some additional properties of the operation Slot defined in (35) are provided; for any $\mathbf{m} \in \mathcal{R}_q$, $\mathbf{m}' \in \mathcal{R}_q$, and $a \in \mathbb{Z}_q^k$ for some $k \in \mathbb{N}$ such that $k \leq \tau$, we have the following properties.

P3) $\text{Slot}(\text{Pack}_k(a) \cdot \mathbf{m}) = \text{Pack}_k(a) \cdot \text{Slot}(\mathbf{m})$.

P4) $\|\text{Pack}_k(a) \cdot \text{Slot}(\mathbf{m})\| \leq \|a^\top\| \cdot \|\text{Slot}(\mathbf{m})\|$.

P5) $\text{UnpackPt}_k(\text{Slot}(\mathbf{m})) = \text{UnpackPt}_k(\mathbf{m})$.

Properties P3–P4 follow from the definition of polynomial multiplication in (1) and the fact that the packing slots are equidistant. P6 is trivial.

By Proposition 2 and properties P2 and P3, it holds that

$$\begin{aligned} & \text{Slot}(\text{Dec}(\sum_{i=0}^{l-1} \text{Enc}'(\text{Pack}_h(K_i)) \boxplus \mathbf{c}_i)) \\ &= \text{Slot}(\sum_{i=0}^{l-1} \text{Pack}_h(K_i) \cdot \text{Dec}(\mathbf{c}_i) + \Lambda) \\ &= \sum_{i=0}^{l-1} \text{Slot}(\text{Pack}_h(K_i) \cdot \text{Dec}(\mathbf{c}_i)) + \text{Slot}(\Lambda) \\ &= \sum_{i=0}^{l-1} \text{Pack}_h(K_i) \cdot \text{Slot}(\text{Dec}(\mathbf{c}_i)) + \text{Slot}(\Lambda) \\ &=: \Omega \in \mathcal{R}_q \end{aligned}$$

for some $\Lambda \in \mathcal{R}_q$ such that $\|\Lambda\| \leq l \cdot \sigma_{\text{Mult}}$. Let $\text{Dec}(\mathbf{c}) = \sum_{j=0}^{N-1} \mathbf{c}_j X^j$. By Lemma 2, $\text{Slot}(\text{Dec}(\mathbf{c}_i)) = \mathbf{c}_{iN/\tau} + \text{Slot}(\Lambda_i)$ for some $\Lambda_i \in \mathcal{R}_q$ such that $\|\text{Slot}(\Lambda_i)\| \leq \log_2 \tau \cdot \sigma_{\text{Mult}}$ for each $i = 0, 1, \dots, l-1$. Hence

$$\begin{aligned} \Omega &= \sum_{i=0}^{l-1} \text{Pack}_h(K_i) \cdot \mathbf{c}_{iN/\tau} + \text{Pack}_h(K_i) \cdot \text{Slot}(\Lambda_i) + \text{Slot}(\Lambda) \\ &=: \sum_{i=0}^{l-1} \text{Pack}_h(K_i) \cdot \mathbf{c}_{iN/\tau} + \bar{\Lambda} \end{aligned}$$

where $\bar{\Lambda} \in \mathcal{R}_q$ is bounded as $\|\bar{\Lambda}\| \leq l(1 + \|K^\top\| \log_2 \tau) \cdot \sigma_{\text{Mult}}$ by P4. Finally, thanks to P5, the left-hand side of (22) is equal to $\text{UnpackPt}_h(\Omega)$. This concludes the proof because $\text{UnpackPt}_h(\sum_{i=0}^{l-1} \text{Pack}_h(K_i) \cdot \mathbf{c}_{iN/\tau}) = K \cdot \text{UnpackPt}_l(\text{Dec}(\mathbf{c}))$ and $\|\text{UnpackPt}_h(\bar{\Lambda})\| \leq \|\bar{\Lambda}\|$.

APPENDIX F PROOF OF LEMMA 3

Applying Proposition 4 to (23a) yields

$$\begin{aligned} \frac{x_0(t+1)}{\text{rsL}} &= \text{UnpackPt}_n(\text{Dec}(\mathbf{x}(t+1))) \\ &= \frac{F\mathbf{x}_0(t)}{\text{rsL}} + \frac{G}{s} \text{UnpackPt}_p(\text{Dec}(\mathbf{y}(t))) + \Delta_1^x(t) \bmod q \end{aligned}$$

for some $\Delta_1^x(t) \in \mathbb{Z}_q^n$ such that

$$\|\Delta_1^x(t)\| \leq \left((n+p) + \left(n \|F^\top\| + \frac{p \|G^\top\|}{s} \right) \log_2 \tau \right) \cdot \sigma_{\text{Mult}}.$$

Since $\lceil y(t)/r \rceil / L = y(t)/(rL) + (\lceil y(t)/r \rceil / L - y(t)/(rL))$, we have

$$\frac{G}{s} \text{UnpackPt}_p(\text{Dec}(\mathbf{y}(t))) = \frac{Gy(t)}{\text{rsL}} + \Delta_2^x(t) \bmod q$$

for some $\Delta_2^x(t) \in \mathbb{Z}_q^n$ such that

$$\|\Delta_2^x(t)\| \leq \frac{\|G\|}{2sL} + \frac{\|G\|}{s} \sigma.$$

Then, with $\Delta^x(t) := \Delta_1^x(t) + \Delta_2^x(t)$, we have

$$\frac{x_0(t+1)}{\text{rsL}} = \frac{F\mathbf{x}_0(t) + Gy(t)}{\text{rsL}} + \Delta^x(t) \bmod q$$

and $\|\Delta^x(t)\| \leq \|\Delta_1^x(t)\| + \|\Delta_2^x(t)\| \leq \alpha'(r, s, L)/\text{rsL}$.

Similarly, applying Proposition 4 to (23b) results in

$$\begin{aligned} \frac{u_0(t)}{\text{rs}^2 L} &= \frac{H}{s} \text{UnpackPt}_n(\text{Dec}(\mathbf{x}(t))) + \Delta^u(t) \bmod q \\ &= \frac{H\mathbf{x}_0(t)}{\text{rs}^2 L} + \Delta^u(t) \bmod q \end{aligned}$$

for some $\Delta^u(t) \in \mathbb{Z}_q^m$ such that

$$\|\Delta^u(t)\| \leq n \left(1 + \frac{\|H^\top\|}{s} \log_2 \tau \right) \cdot \sigma_{\text{Mult}} = \frac{\beta'(r, s, L)}{\text{rs}^2 L}.$$

Finally, it can be easily derived by applying Proposition 1 that

$$\frac{x_0(0)}{\text{rsL}} = \frac{x^{\text{ini}}}{\text{rsL}} + \Delta^{\text{ini}} \bmod q$$

for some $\Delta^{\text{ini}} \in \mathbb{Z}_q^n$ such that $\|\Delta^{\text{ini}}\| \leq \gamma(r, s, L)/\text{rsL}$. From this point, the rest of the proof is analogous to that of Lemma 1 and is thus omitted.

REFERENCES

- [1] H. Sandberg, S. Amin, and K. H. Johansson, "Cyberphysical security in networked control systems: An introduction to the issue," *IEEE Control Syst. Mag.*, vol. 35, no. 1, pp. 20–23, Feb. 2015.
- [2] A. Teixeira, I. Shames, H. Sandberg, and K. H. Johansson, "A secure control framework for resource-limited adversaries," *Automatica*, vol. 51, pp. 135–148, 2015.
- [3] K. Kogiso and T. Fujita, "Cyber-security enhancement of networked control systems using homomorphic encryption," in *Proc. 54th IEEE Conf. Decis. Control*, 2015, pp. 6836–6843.
- [4] M. Schulze Darup, A. B. Alexandru, D. E. Quevedo, and G. J. Pappas, "Encrypted control for networked systems: An illustrative introduction and current challenges," *IEEE Control Syst. Mag.*, vol. 41, no. 3, pp. 58–78, Jun. 2021.
- [5] J. Kim, D. Kim, Y. Song, H. Shim, H. Sandberg, and K. H. Johansson, "Comparison of encrypted control approaches and tutorial on dynamic systems using learning with errors-based homomorphic encryption," *Annu. Rev. Control*, vol. 54, pp. 200–218, 2022.
- [6] O. Regev, "On lattices, learning with errors, random linear codes, and cryptography," *J. ACM*, vol. 56, no. 6, 2009, Art. no. 34.
- [7] C. Gentry, "Fully homomorphic encryption using ideal lattices," in *Proc. 41st Annu. ACM Symp. Theory Comput.*, 2009, pp. 169–178.
- [8] J. Kim, H. Shim, and K. Han, "Dynamic controller that operates over homomorphically encrypted data for infinite time horizon," *IEEE Trans. Autom. Control*, vol. 68, no. 2, pp. 660–672, Feb. 2023.

- [9] C. Gentry, A. Sahai, and B. Waters, "Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based," in *Proc. 33rd Annu. Cryptol. Conf.*, 2013, pp. 75–92.
- [10] V. Lyubashevsky, C. Peikert, and O. Regev, "On ideal lattices and learning with errors over rings," *J. ACM*, vol. 60, no. 6, 2013, Art. no. 43.
- [11] SEAL Microsoft (Release 4.1), Microsoft Research, Redmond, WA, USA, Jan. 2023. [Online]. Available: <https://github.com/Microsoft/SEAL>
- [12] S. Halevi and V. Shoup, "Design and implementation of HELib: A homomorphic encryption library," *Cryptol. ePrint Arch.*, Paper 2020/1481, 2020. [Online]. Available: <https://eprint.iacr.org/2020/1481>
- [13] Lattigo v6, EPFL-LDS, Tune Insight SA, Lausanne, Switzerland, Aug. 2024. [Online]. Available: <https://github.com/tuneinsight/lattigo>
- [14] K. Teranishi, T. Sadamoto, and K. Kogiso, "Input-output history feedback controller for encrypted control with leveled fully homomorphic encryption," *IEEE Trans. Control Netw. Syst.*, vol. 11, no. 1, pp. 271–283, Mar. 2024.
- [15] J. Lee, D. Lee, J. Kim, and H. Shim, "Encrypted dynamic control exploiting limited number of multiplications and a method using RLWE based cryptosystem," *IEEE Trans. Syst., Man, Cybern. Syst.*, vol. 55, no. 1, pp. 158–169, Jan. 2025.
- [16] I. Chillotti, N. Gama, M. Georgieva, and M. Izabach  ne, "Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, 2016, pp. 3–33.
- [17] M. R. Albrecht et al., "Homomorphic encryption standard," in *Protecting Privacy Through Homomorphic Encryption*, K. Lauter, W. Dai, and K. Laine, Eds. Cham, Switzerland: Springer, 2021, pp. 31–62.
- [18] C. Gentry, S. Halevi, and N. P. Smart, "Homomorphic evaluation of the AES circuit," in *Proc. 32nd Annu. Cryptol. Conf.*, 2012, pp. 850–867.
- [19] J. Kim, H. Shim, H. Sandberg, and K. H. Johansson, "Method for running dynamic systems over encrypted data for infinite time horizon without bootstrapping and re-encryption," in *Proc. 60th IEEE Conf. Decis. Control*, 2021, pp. 5614–5619.
- [20] M. S. Tavazoei, "Non-minimality of the realizations and possessing state matrices with integer elements in linear discrete-time controllers," *IEEE Trans. Autom. Control*, vol. 68, no. 6, pp. 3698–3703, Jun. 2023.
- [21] J. Lee, D. Lee, and J. Kim, "Stabilization by controllers having integer coefficients," 2025, *arXiv:2505.00481*.
- [22] D. A. Dowler, "Bounding the norm of matrix powers," M.S. thesis, Dept. Math., Brigham Young Univ., Provo, UT, USA, 2013.
- [23] K. H. Johansson, "The quadruple-tank process: A multivariable laboratory process with an adjustable zero," *IEEE Trans. Control Syst. Technol.*, vol. 8, no. 3, pp. 456–465, May 2000.



Yeongjun Jang received the B.S. degree in electrical and computer engineering in 2022 from Seoul National University, Seoul, South Korea, where he is currently working toward the combined M.S./Ph.D. degree in electrical and computer engineering.

His research interests include data-driven control and encrypted control systems.



Joowon Lee received the B.S. degree in electrical and computer engineering in 2019 from Seoul National University, Seoul, South Korea, where she is currently working toward the combined M.S./Ph.D. degree in electrical and computer engineering.

Her research interests include data-driven control, encrypted control systems, and cyber-physical systems.



Seonhong Min received the B.S. degree in mathematical sciences in 2022 from Seoul National University, Seoul, South Korea, where he is currently working toward the combined M.S./Ph.D. degree in computer science and engineering.

His research interests include fully homomorphic encryption and its applications, such as privacy preserving machine learning and multiparty computation.



Hyesun Kwak received the B.S. degrees in mathematics education and computer science and engineering both in 2021 and the M.S. degree in computer science and engineering in 2023 all from Seoul National University, Seoul, South Korea.

She is currently a Software Engineer with Samsung Electronics, Suwon, South Korea. Her research interests include homomorphic encryption and its applications, such as privacy-preserving machine learning.



Junsoo Kim received the B.S. degrees in electrical engineering and mathematical sciences both in 2014 and the M.S. and Ph.D. degrees in electrical engineering in 2020, all from Seoul National University, Seoul, South Korea.

He was a Postdoctoral Researcher with the KTH Royal Institute of Technology, Stockholm, Sweden, till 2022. He is currently an Assistant Professor with the Department of Electrical and Information Engineering, Seoul National University of Science and Technology, Seoul. His research interests include security problems in networked control systems and encrypted control systems.



Yongsoo Song received the B.S. degree in mathematical sciences in 2012 and the M.S. and Ph.D. degrees in mathematical sciences in 2018, all from Seoul National University, Seoul, South Korea.

He was a Postdoctoral Researcher with the University of California San Diego, San Diego, CA, USA, till 2019. From 2019 to 2021, he was a Senior Researcher with Cryptography and Privacy Research Group, Microsoft Research Redmond, Redmond, WA, USA. He is currently an

Assistant Professor with the Department of Computer Science and Engineering, Seoul National University. His research interests include in privacy-enhancing cryptography for secure computation, which includes fully homomorphic encryption, multiparty computation, and zero-knowledge proofs, approached from both theoretical and applied perspectives.